

Questions on OOPS

1. Design a banking system with:

- An abstract base class `Account` with `deposit()`, `withdraw()`, `calculate_interest()`.
- Subclasses: `SavingsAccount`, `CurrentAccount`, `FixedDepositAccount`.
- Each account must:
 - Encapsulate balance (private)
 - Provide controlled access through properties
 - Override interest calculation differently
- Include a static method to validate amount.
- Include a class method to update bank-wide interest policies.

Demonstrate:

- Polymorphic behavior by iterating through all account types
- Preventing direct access to balance
- Multiple interest strategies

2. Build:

- `Vehicle` base class
- `Car`, `Bike`, `Auto` subclasses
- A `Driver` class that **contains** a `Vehicle`
- A `Ride` class that:
 - Calculates fare differently depending on the type of vehicle (polymorphism)
 - Stores driver + vehicle combination
 - Protects internal fare formula through encapsulation

Also:

- Use `__str__` to print readable ride summaries.

Show how composition + polymorphism interact.

3. Create:

- Abstract class `PaymentMethod` with `pay()`, `validate()`
- Subclasses: `CardPayment`, `WalletPayment`, `UPIPayment`
- Encapsulate user balance
- Use `@property` to control reading available funds
- Overload + operator to combine two payment methods into “split payment”
- Demonstrate polymorphism through a checkout loop.

4. Create classes:

- `Person` → `base`
- `MedicalStaff(Person)`
- `Doctor(MedicalStaff)`
- `Surgeon(Doctor)`

Requirements:

- Hide sensitive data (e.g., salary, patient notes)
- Abstract method `perform_duty()`
- Each level overrides the method with more specific behavior
- Use `super()` to chain constructor calls

Demonstrate consistency across hierarchy.

5. Classes:

- `User`
- `Instructor(User)`
- `Student(User)`
- `TeachingAssistant(Student, Instructor)`

Requirements:

- Track course assignments privately
- Ensure TAs override `submit_work()` and `grade_work()`
- Print MRO and explain how Python resolves conflicts

6. Create:

- `Product` class with private price and quantity
- `Warehouse` class containing multiple products
- Overload:
 - `+` to merge warehouses
 - `len()` to return number of unique products
 - `in` operator to check if product exists
- Provide class method to track total warehouses created

7. Design:

- Abstract class `MediaFile` with `play()`, `stop()`
- Subclasses: `MP3File`, `MP4File`, `WAVFile`
- Private file path validation done internally
- A function `start_player(media)` that works with ANY object that has `play()` (duck typing)

Demonstrate mixing true polymorphism + duck typing.

8. Create:

- Abstract class `StatementFormatter`
- Subclasses: `PDFFormatter`, `JSONFormatter`, `TextFormatter`
- Overload `__call__()` so that formatters can be used like functions
- Overload `__repr__` for debugging
- Demonstrate polymorphic behavior in a reporting pipeline

9. Classes:

- `LightDevice`
- `SecurityDevice`
- `SmartCamera(LightDevice, SecurityDevice)`

Requirements:

- Resolve method conflicts using MRO
- Encapsulate internal camera logs
- `SmartCamera` overrides both parents' behaviors
- Use `super()` responsibly in multiple inheritance

10. Create:

- Abstract class `MenuItem` with `get_price()`
- Subclasses: `Pizza`, `Burger`, `Drink`
- `Order` class containing a list of items (composition)
- Encapsulate the list internally
- Override methods to apply custom pricing logic for each food type

11. Create:

- Class `Applicant` with private skills list
- Overload:
 - `+` to add skill
 - `-` to remove skill
 - `==` to compare applicants who have identical skill sets
- Use inheritance to create `ExperiencedApplicant` with additional fields

12. Create:

- `Character` → base class
- `Warrior`, `Archer`, `Mage` subclasses

Each subclass:

- Overrides `attack()`
- Encapsulates health with `@property`

- Prevents negative HP
- Uses class attributes for shared attributes (e.g., stamina_cost)

Demonstrate polymorphic combat simulation.

13. Build:

- Transport abstract class
- Subclasses: Taxi, Bus, Train
- Each implements:
 - calculate_fare() differently
- Use static method to validate distance
- Encapsulate fare state
- Add class method to update government tax slab

14. Design:

- Abstract class Model with train(), predict()
- Implement LinearRegressionModel and DecisionTreeModel (just print or write a logic, focus on calling and concept)
- A Pipeline class that:
 - Accepts any model
 - Uses composition to chain transformations
 - Overloads __call__() to run predictions
- Encapsulates internal steps

15. Create a mini version of Amazon with:

Classes:

- User
- Product
- Seller(User)
- Buyer(User)
- Order
- Cart

Requirements (must use all OOP concepts):

>Inheritance: Seller and Buyer extend User
 >Encapsulation: protect internal cart list, user password
 >Abstraction: base class User defines abstract get_role()
 >Polymorphism: different users behave differently in checkout
 >Composition: Buyer “has” a Cart
 >Operator overloading:

- + to add product to Cart
- - to remove product
- >Properties: validate product price

- >Class methods: tracking total users
- >Static methods: validating product IDs
- > `__str__` for readable summaries
- >MRO behavior when Buyer inherits from multiple mixins (e.g., `RewardsMixin`)